

Advance Function

Closure in Python

A closure is a function that remembers variables from its outer scope even after the outer function has finished execution.

A closure is a function object that remembers values from its enclosing (lexical) scope even when that scope is no longer active.

Python implements closures using cell objects that store free variables referenced by nested functions.

```
def outer():  
    x = 10  
  
    def inner():  
        return x  
  
    return inner  
  
f = outer()  
print(f())
```

Why is this closure?

Because after *outer()* finished execution *x* should normally disappear, but *f()* still remembers the value of *x* that remembered state is called a closure.

What Actually Happens in Memory?

- Python uses:
- Stack Frames
- Heap Objects
- **Cell Objects (for closures)**

A cell object is a small container object that holds a reference to a variable used in a closure.

It exists so that multiple function can share the same variable.

The variable can survive after the outer function finishes.

```
def outer():  
    x = 10  
  
    def inner():  
        return x  
  
    return inner  
  
f = outer()  
  
print(f.__closure__)  
print(f.__closure__[0].cell_contents)
```

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal x  
  
        x += 1  
        return x  
  
    return inner  
  
# outer() returns inner()  
f = outer()  
  
print(f())    # 11  
print(f())    # 12  
print(f())    # 13  
print(f())    # 14
```

Lexical (enclosing) Scope

Lexical Scope means a function resolves variables according to where the function is defined in the source code, not where it is called from during execution.

inner() can access x because x belongs to its enclosing scope. This is called Lexical Scoping.

```
x = 100

def outer():
    x = 10

    def inner():
        return x

    return inner

f = outer()

print(f())
```

Higher Order Functions (HOF)

- A Higher Order Function is a function that:
 1. Takes another function as an argument
OR
 2. Returns a function
OR
 3. Both

Why is this possible?

Because functions in Python are First-Class Objects.

Meaning:

- Functions can be:
- Stored in variables
- Passed as arguments
- Returned from functions
- Stored inside data structures
- Have identity, type and memory address

Passing Function

```
def operate(func, x, y):  
    return func(x, y)  
  
def add(a, b):  
    return a + b  
  
operate(add, 5, 3)
```

You are NOT passing code.

You are passing a reference to a function object in memory.

Why HOF Exists

- **Abstraction**
 - Separate what to do from how to apply it.
- **Reusability.**
 - Write generic logic once
- **Behavior Parameterization.**
 - Instead of passing data, pass behavior.

Lazy Evaluation Using map()

Python creates a map object.

- It stores:
- Function reference
- Iterable reference
- But it does not calculate anything.
- Only when forced: list(m).

```
numbers = [1, 2, 3, 4]

m = map(lambda x: x * x, numbers)

print(m)
```

What makes HOF powerful?

HOF is not about map(), map() is just an example.

- The real idea is instead of writing fixed logic you pass logic as a parameter.

```
def square_list(numbers):  
    result = []  
  
    for n in numbers:  
        result.append(n*n)  
  
    return result
```

```
def apply_op(numbers, func):  
  
    result = []  
  
    for n in numbers:  
        result.append(func(n))  
  
    return result
```

```
apply_op([1,2,3], lambda x: x*x)  
  
apply_op([1,2,3], lambda x: x*x*x)
```

Iterable

An iterable is an object capable of returning its members one at a time.

In simple words:

An iterable is any object that can be used inside a: for loop

- list
- tuple
- string
- set
- dictionary
- range

How Does Python Know It Is Iterable?

- Because internally it implements: `__iter__()`

```
nums = [1, 2, 3]

print(dir(nums))
```

Iterator

An iterator is an object that: Remembers current position

Produces one value at a time

Implements `__next__()`

- Iterable: Collection
- Iterator: Producer

```
nums = [1, 2, 3]
```

```
it = iter(nums)
```

```
print(next(it))
```

```
print(next(it))
```

```
print(next(it))
```

```
nums = [1,2,3]
```

```
for n in nums:  
    print(n)
```

```
it = iter(nums)
```

```
while True:
```

```
    try:
```

```
        value = next(it)
```

```
        print(value)
```

```
    except StopIteration:
```

```
        break
```


Iterable vs Iterator

Iterable	Iterator
Can be looped over	Produces values
Has <code>iter()</code>	Has <code>next()</code>
Example: list	Example: list_iterator
Doesn't remember position	Remembers position
Needs <code>iter()</code>	Uses <code>next()</code>

Custom Iterator

Python allows us to create our own iterators.

Think of `__iter__()` as answering this question:

"If someone wants to iterate (loop) over me, which iterator should I use?"

I am already an iterator. Use me.

```
c = Counter(3)

it = iter(c)

print(it is c)
```

```
class Counter:
```

```
    def __init__(self, max_value):
        self.max_value = max_value
        self.current = 0
```

```
    def __iter__(self):
        return self
```

```
    def __next__(self):

        if self.current < self.max_value:
            self.current += 1
            return self.current

        raise StopIteration
```

```
c = Counter(3)
```

```
for x in c:
    print(x)
```

Generator

Generator is a special type of iterator created using: `yield`

Generator automatically provides: `__iter__()` and `__next__()` no need to write them manually.

Normal function

Return everything together

```
def demo():  
    return [1, 2, 3]
```

Generator function

Produces values one at a time.

```
def demo():  
  
    yield 1  
    yield 2  
    yield 3
```

```
g = demo()  
  
print(next(g))  
print(next(g))  
print(next(g))
```

What Does yield actually mean?

Think → yield = pause + return value

Unlike → return which terminates the function.

Yield → pauses execution.

Why Generators are powerful?

range(1000000000000)

A list would consume huge memory.

Generator.

Stores only one value at a time.

```
def big():  
  
    for i in range(10000000000):  
        yield i
```

yield from

Used to delegate work to another generator.

```
def gen2():  
  
    for value in gen1():  
        yield value
```

```
def gen2():  
    yield from gen1()
```

```
def gen1():  
    yield 1  
    yield 2  
  
def gen2():  
    yield from gen1()  
    yield 3  
  
for x in gen2():  
    print(x)
```

Operation	Time Complexity
Create generator	$O(1)$
<code>next(generator)</code>	$O(1)$
Iterate through all n values	$O(n)$
Convert to list	$O(n)$
Space used by generator	$O(1)$